

8교시: 차주 수업내용 Overview

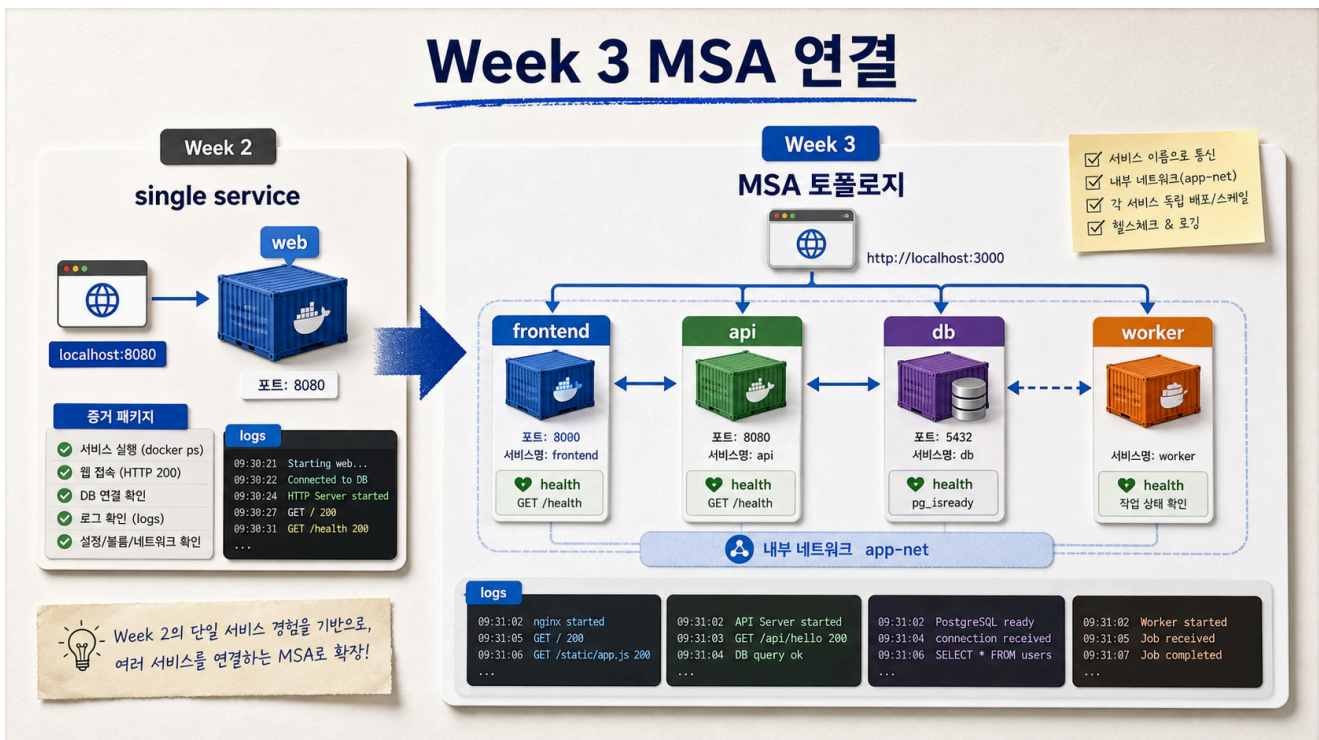
수업 목표

- Week 3 MSA가 Week 2 Docker/Compose 경험에서 어떻게 확장되는지 설명한다.
- 단일 service에서 multi-service로 넘어갈 때 늘어나는 운영 복잡도를 예측한다.
- Week 3 readiness checklist를 작성한다.

50분 흐름

시간	활동	비중	산출
0-8분	Week 2 마감 정리	설명 15%	final note
8-20분	단일 service에서 MSA로 확장	설명 25%	topology preview
20-32분	Week 3 운영 질문	활동 25%	readiness checklist
32-42분	Docker/Compose 개념 재사용	설명 20%	concept bridge
42-50분	다음 액션 정리	실행 15%	Week 3 prep

Visual 1: Week 3 MSA 연결



이 visual은 Day 5의 단일 Docker app이 Week 3의 frontend, api, db, worker topology로 확장되는 흐름을 보여준다.

핵심 설명

Week 2는 Docker로 실행 환경을 표준화하는 주간이었다. Week 3는 여러 service가 함께 동작할 때 인프라 운영자가 무엇을 봐야 하는지 다룬다.

단일 container에서는 port, config, log, data가 하나의 app에 모여 있다. MSA에서는 service마다 port, config, log, health, dependency가 생긴다. 복잡도는 단순히 service 수만큼 더해지는 것이 아니라 연결 관계 때문에 더 빠르게 증가한다.

Week 2에서 Week 3로 이어지는 개념

Week 2	Week 3 확장
image	service별 image
container	service instance
port binding	frontend/API external endpoint
Compose network	service-to-service communication
environment	service config and dependency URL
volume	database persistence
logs	distributed logs
healthcheck	liveness/readiness
README handoff	topology/runbook

MSA에서 새로 커지는 질문

질문	이유
어떤 service가 외부에 노출되는가	attack surface와 routing
service 간 host name은 무엇인가	DNS/service discovery
API가 DB 준비 전 시작하면?	readiness와 retry
장애가 어디까지 전파되는가	dependency failure
로그는 어디서 모으는가	distributed observability
데이터 소유권은 어디인가	coupling과 migration

readiness checklist

```
## Week 3 Readiness
- Dockerfile build 가능:
```

- Compose 실행 가능:
- service name과 localhost 차이 설명 가능:
- HTTP status 확인 가능:
- logs 확인 가능:
- env/secret 분리 설명 가능:
- volume data lifecycle 설명 가능:
- 남은 blocker:

실무 insight

MSA는 항상 좋은 선택이 아니다. 서비스가 나뉘면 독립 배포와 팀 경계는 좋아질 수 있지만, 네트워크 장애, 버전 호환성, observability, data consistency가 어려워진다. Week 3에서는 MSA를 유행어가 아니라 운영 복잡도를 감당할 이유가 있을 때 선택하는 구조로 다룬다.

학술 기준 연결

이 교시는 transfer learning이다. Week 2에서 배운 개념을 새로운 문제 공간인 MSA에 적용한다. 학생은 예제 명령을 외우는 것이 아니라 port, network, config, log, health 같은 개념을 다른 구조에 옮겨야 한다.

오해 점검

오해	교정
MSA는 무조건 좋은 구조다	운영 복잡도와 trade-off가 있다
Compose service name만 알면 MSA 끝이다	dependency, health, logs, data가 추가된다
Week 2가 끝나면 Docker는 끝이다	Week 3 실습의 실행 기반이다
API 장애는 개발자만 본다	인프라 관점의 evidence와 routing도 중요하다

Week 3 질문 템플릿

My Week 3 Question

- 단일 Docker app에서는 쉬웠던 것:
- 여러 service가 되면 어려워질 것:
- 내가 확인하고 싶은 evidence:
- 내가 걱정하는 failure mode:

평가 기준

기준	2점 evidence
연결	Week 2 개념을 Week 3 service 구조로 mapping했다
복잡도	MSA의 운영 trade-off를 설명했다
준비	readiness checklist를 작성했다
질문	구체적인 Week 3 질문을 남겼다

마무리 문장

Week 2에서 우리는 한 서비스를 재현 가능하게 실행하는 법을 배웠다. Week 3에서는 여러 서비스가 서로 의존할 때 실행, 연결, 관찰, 장애 대응이 어떻게 어려워지는지 배운다.

공식 근거 링크

- Docker Compose networking: <https://docs.docker.com/compose/how-tos/networking/>
- Google SRE Book: <https://sre.google/sre-book/table-of-contents/>

Week 3 사전 용어

용어	Week 2에서의 연결
frontend	browser가 접근하는 web service
api	HTTP request를 처리하는 backend service
database	persistent data dependency
worker	background process
service discovery	Compose service name과 연결
health check	Day 5 healthcheck에서 확장
dependency failure	DB readiness 문제에서 확장

Week 3 readiness self-score

항목	0	1	2
Compose	실행 어려움	실행 가능	config 해석 가능
Network	localhost만 이해	service name 사용	경계 설명 가능
Logs	단일 logs만 봄	service logs 봄	장애 분류 가능
Health	모름	HTTP status 확인	liveness/readiness 질문 가능
Config	image에 넣음	env 사용	secret 분리 설명
Handoff	없음	명령 기록	runbook 형태

topology preview

```
browser -> frontend -> api -> database
                        -> worker
```

운영자는 각 화살표를 질문으로 바꾼다.

화살표	질문
browser -> frontend	외부 port와 URL은 무엇인가
frontend -> api	API base URL은 어떻게 주입되는가
api -> database	credential과 migration은 어떻게 관리하는가
api -> worker	비동기 작업과 실패는 어디서 보나

Week 3 위험 예측

Risk Prediction

- 서비스가 늘면 어려워질 것:
- 가장 먼저 볼 evidence:
- 내가 헛갈릴 것 같은 개념:
- 수업 전에 복습할 Week 2 자료:

Lesson 8 Exit Ticket

Exit Ticket

- Week 2에서 가장 중요한 Docker 개념:
- Week 3에서 재사용할 개념:

- MSA에서 커질 운영 위험:
- 첫 번째로 확인하고 싶은 질문:

Week 2 종료 기준

Week 2는 Docker 명령 암기로 끝나지 않는다. 학생이 단일 service의 실행 계약을 설명하고, 그 계약이 여러 service로 확장될 때 어떤 운영 문제가 커지는지 질문할 수 있으면 Week 3로 넘어갈 준비가 된 것이다.

마감 확인:

- 단일 service의 build/run/check를 설명할 수 있는가?
- service name과 localhost의 차이를 설명할 수 있는가?
- 여러 service의 logs와 health가 분산될 것을 예상하는가?

Week 3 사전 용어

용어	Week 2에서의 연결
frontend	browser가 접근하는 web service
api	HTTP request를 처리하는 backend service
database	persistent data dependency
worker	background process
service discovery	Compose service name과 연결
health check	Day 5 healthcheck에서 확장
dependency failure	DB readiness 문제에서 확장

topology preview

```
browser -> frontend -> api -> database
                -> worker
```

운영자는 각 화살표를 질문으로 바꾼다.

화살표	질문
browser -> frontend	외부 port와 URL은 무엇인가
frontend -> api	API base URL은 어떻게 주입되는가
api -> database	credential과 migration은 어떻게 관리하는가
api -> worker	비동기 작업과 실패는 어디서 보나

Week 2 복습 과제

- docker compose config를 읽을 수 있다.
- localhost와 service name을 구분할 수 있다.
- HTTP 200과 body marker를 확인할 수 있다.
- logs를 service별로 볼 수 있다.
- volume 삭제 위험을 설명할 수 있다.
- secret을 image에 넣으면 안 되는 이유를 말할 수 있다.

Week 3 준비 질문 예시

Prepared Question

- If api starts before database is ready, what evidence should we check?
- How does frontend know api URL?
- Which service should be exposed to host?
- How do we collect logs from multiple services?